Receipt to Expense

A fine-tuned Gemma 4 E2B that reads receipts and splits the bill

Team Come_Join_Qarc · SSIT Tumkur · Theme 3B · Hacktoberfest Hack Day · 24 Oct 2026

Lead: Mohammed Owais **Members:** Mohammed Muzakkir · Mohammad Tariq Afzal · Mohammed Ayaan · Mohammed Younus

66 %
all four fields right
(prompted Gemma: 37 %)

2.5 min
LoRA training on
one GPU (626 receipts)

5/5 + 8/8
deliverables +
bonus items

Adapters + model card
huggingface.co/owaisnoe/
gemma4-e2b-sroie-lora

Gemma 4 E2B + LoRA (Unsloth) · SROIE receipts · Gradio app with a public link

Problem: receipts are still typed in by hand

SAAM · BHED

Who: students sharing meals and trips, and small shops logging purchases.
Typing receipts in by hand is slow and error-prone.

Today's expense apps

built for companies, not for students

- Closed, paid, cloud OCR
- Built for company cards
- Never say which field to trust

Receipt to Expense

what we built today

- Free, open 2B-class Gemma 4 E2B you can self-host + one LoRA file
- Proven: baselines, significance tests, 3 datasets
- Says which field to double-check, and splits the bill

No OCR step. Most receipt hacks chain OCR + an LLM; ours is one fine-tuned open vision-language model from photo to fields, measured with McNemar and out-of-distribution tests.

Data and method

dataset description and training settings

Data: SROIE (ICDAR 2019, CC BY 4.0)

- 626 train / 361 test scanned receipts
- Labels: company, date, address, total
- Test set: fixed 100 receipts (seed 0); keys asserted disjoint from training
- Never trained on test, external or local receipts

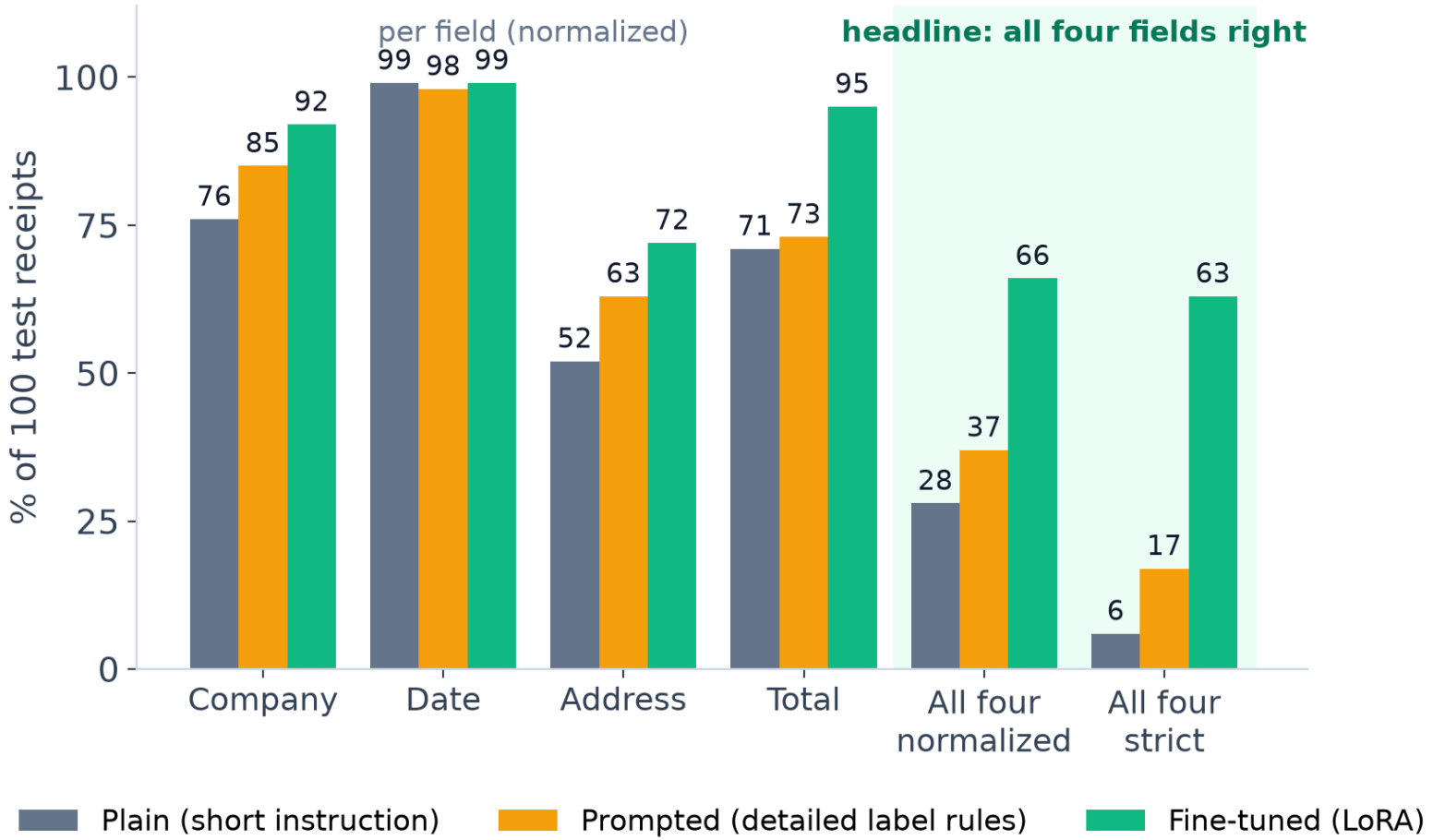
Method: LoRA on Gemma 4 E2B

- LoRA r = 32 on vision + language layers: 59.8 M trainable (1.15 %)
- 5 smoke + 157 steps = 1 epoch: 2.5 min on one G4 GPU, peak 13 GB
- Image first, 560 visual tokens (525-544 measured)
- Greedy decoding, batch size 1, for every configuration

Receipt photo → Gemma 4 E2B + LoRA image first, 560 tokens → JSON: company, date, address, total → App: check, split, add to expenses

Fine-tuning beats prompting on every field

DAND



66 %

all four fields, normalized
prompted 37 % · plain 28 %

63 %

all four fields, strict
prompted 17 % · plain 6 %

p = 1e-6

McNemar vs prompted:
33 receipts better, 4 worse

Same 100 test receipts, same 560-token image budget, greedy batch 1. Normalized forgives case, spacing, currency and date format, so it measures reading, not formatting.

Live demo: snap a receipt, check it, split it

DAAM

working application built around the fine-tuned model

1 Snap or upload

phone camera, webcam or a file

2 Fine-tuned fields

editable; plain model beside it; trust check: "agree on N/4 fields, double-check X"

3 Known-vendor badge

vendor memory from the training labels (RAG)

4 Split the bill

equally or item by item; items auto-filled by zero-shot line items; shared by names or 'all'

5 Group balances

tax, service and rounding shared in proportion; settle up with the fewest payments

6 Expenses + CSV

running total of expenses; download the ledger as CSV

Try it on your phone

f6753b53a276c78344.gradio.live

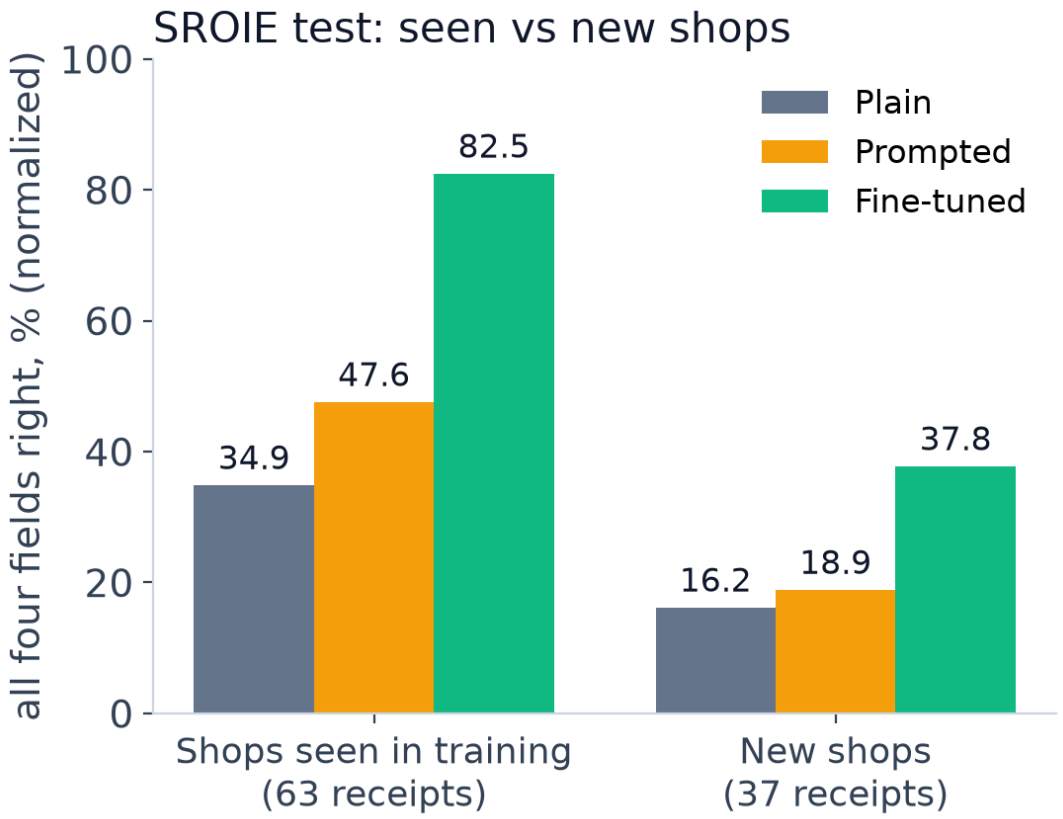
Judges: lend us a receipt!
Extract about 7 s (fine-tuned + plain); "please check" when a date or total does not parse

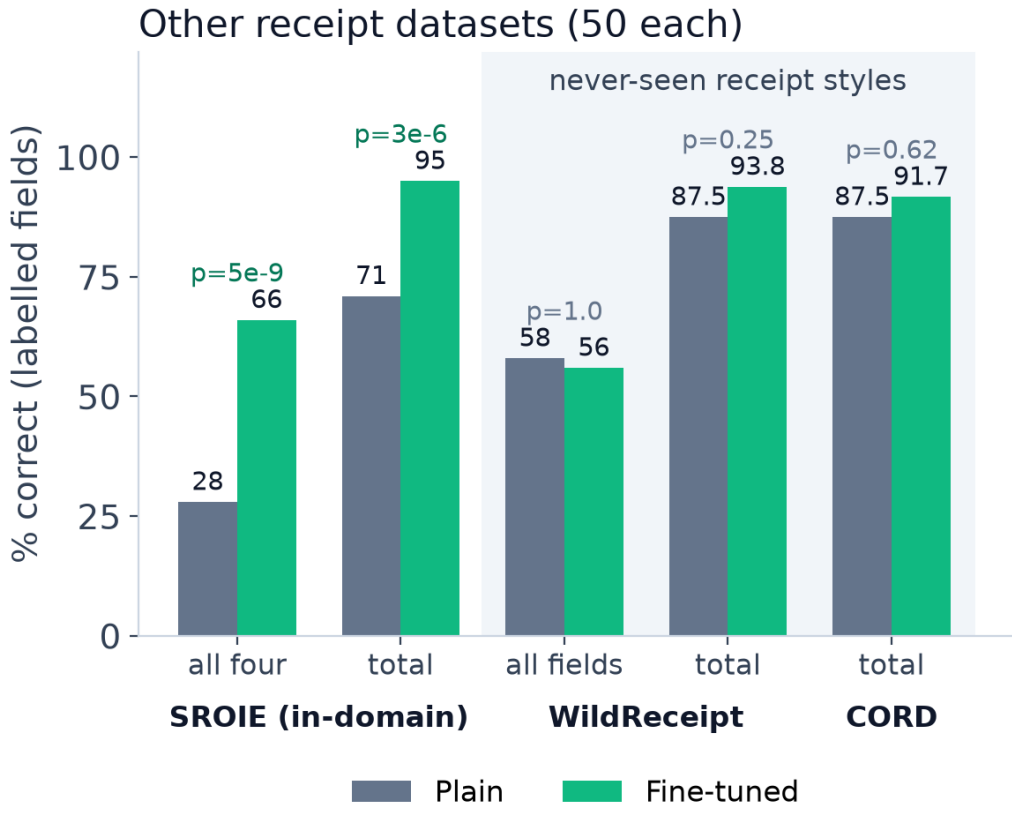
Open model: Gemma 4 E2B (2B-class) + one LoRA adapter file: no cloud OCR, no subscription.
Built for students: split equally or by item, tax shared fairly, everyone settled in the fewest payments.

Wins on new shops too, and stays safe on other receipts

DAND

where fine-tuning helps, its limits, and why





Helps most: totals (73 -> 95 %), addresses and receipt conventions; on brand-new shops it still doubles prompting (38 vs 19 %). **Next frontier:** other countries' receipts, safe today (totals up on all 3 datasets); Indian receipts next.

Delivered: 5 / 5 required, 8 / 8 bonus

DELIVERED

Bonus item	Status	Evidence
Prompted baseline	Done	37 % vs fine-tuned 66 % (p = 1e-6)
Training experiments	Done	budget 280 / 560 / 1120, rank 8, 2 epochs, curve
Overfitting check	Done	WildReceipt + CORD: no loss; wins on new shops
Model size (E4B)	Done	69 vs 66 % (n.s.); 3.46 vs 3.0 s; 17.4 vs ~11 GB
Combined with RAG	Done	vendor memory: 66 -> 69 %, prompted 37 -> 47 %
Publishing on HF	Done	adapters + model card on Hugging Face
Error analysis	Done	9 categories, fixes proposed
3B: line items	Done (zero-shot)	CORD item F1 71 -> 79 %

Required: 5 / 5 ✓

- Fine-tuned model + settings (HF)
- Dataset description (slide 3)
- Comparison + examples (slide 4)
- Working app, live (slide 5)
- Helps / limits / why (slide 6)

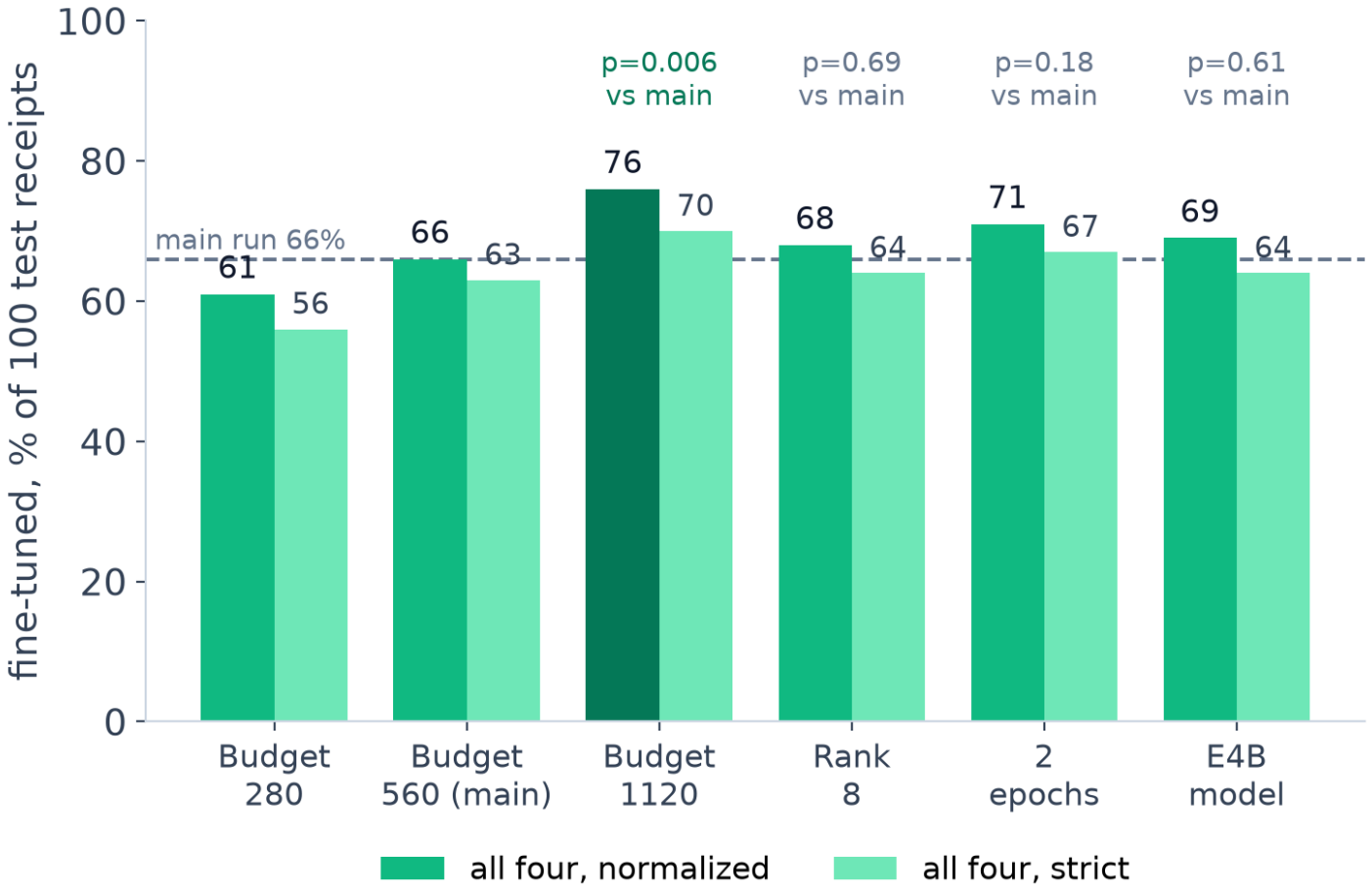
Next

- Retrain at 1120 visual tokens (76 %)
- Indian receipts + local photos
- UPI payment requests from the settle-up list
- On-device 4-bit E2B: receipts stay on the phone
- Calibrated trust score (totals)

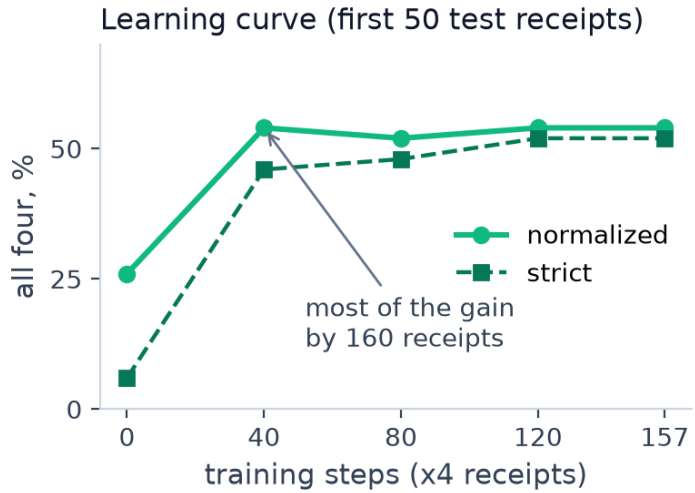
BACKUP

What moves accuracy: resolution, not size

training experiments + model size, same 100 test receipts



- 1120 tokens: 76 % vs 66 % (p = 0.006)
- Rank 8 = rank 32 (68 vs 66 %, n.s.)
- 2 epochs: 71 % (n.s.)
- E4B: 69 % (n.s.); 3.46 vs 3.0 s per receipt, 17.4 vs ~11 GB peak



Each budget arm is trained and tested at its own budget. Plain gets worse with more visual tokens (33 -> 28 -> 22 %) while fine-tuned gets better (61 -> 66 -> 76 %): an observation, not yet explained.

Trust check: which field to double-check

BACKUP

fine-tuned vs plain agreement, measured on the 100 SROIE test receipts

Field	Models agree	Fine-tuned right when they agree	Fine-tuned right when they differ	Errors flagged by "differ"
Company	76 %	97 %	75 %	75 %
Date	99 %	100 %	0 %	100 %
Address	59 %	83 %	56 %	64 %
Total	72 %	96 %	93 %	40 %

29 %

of receipts: all four fields agree

83 % vs 59 %

all four right when they agree
vs when any field differs

Useful but imperfect. A disagreement flags 3 in 4 company errors and about 2 in 3 address errors, but only 40 % of total errors: both models can agree on the same wrong total. The app shows it as "agree on N/4 fields; double-check X".

Error analysis: most misses are one-character slips

BACKUP

fine-tuned model, 100 test receipts

Category	Company	Date	Address	Total
correct	91	99	71	92
format only	1	0	1	3
punctuation / spacing	3	0	9	0
near miss (1-2 chars)	2	0	11	0
OCR-confusable (8/B)	0	0	3	0
extra text	2	0	1	0
truncated	0	0	1	0
rounding (<= 0.05)	0	0	0	3
different value	1	1	3	2

Address: gold ...B1750 MASAI
read **...81750 MASAI**
label noise: 81750 is Masai's real postcode

Total: gold 7.42
read **RM7.40**
rounding adjustment printed on the receipt

Total: gold 6.40
read **6.04**
digit swap

Company: gold THONG RECIPE
read **16G BLOCK B, ZENITH CORPORATE PARK**
read the address line as the name

Fixes: punctuation-tolerant address matching · vendor memory (RAG) · higher visual budget (1120: 76 %)

What we found under the hood

BACKUP

understanding how it works: a hidden bug caught and fixed

A silent resolution bug (fixed)

- Unsloth's collator shrank receipts to 512 px and snapped them to 32 px
- Token count identical (525 = 525), pixels not: max difference 0.68
- Fix: images column + resize="max": difference 0.0000

Known SDPA bug: checked

- torch <= 2.13 corrupts prompts of length 1 mod 32
- 0 / 100 of our prompts affected (both instructions)
- Eager attention matched SDPA exactly on 3 + 3 receipts

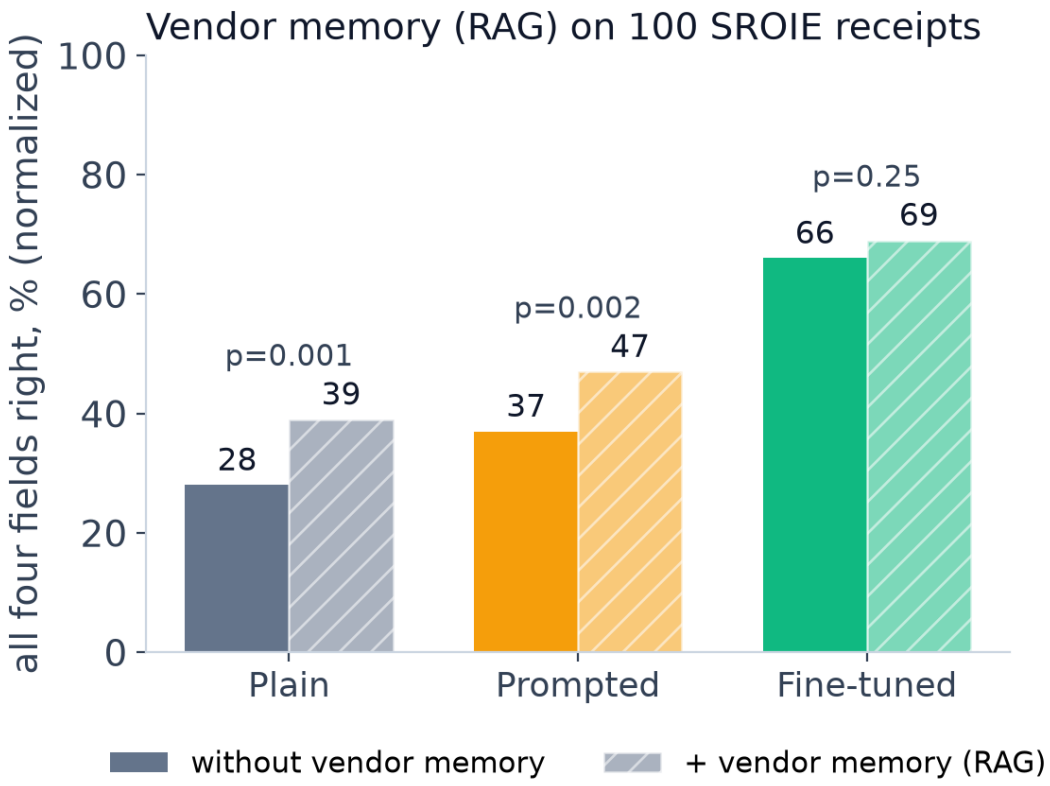
80 of 718 LoRA tensors never trained

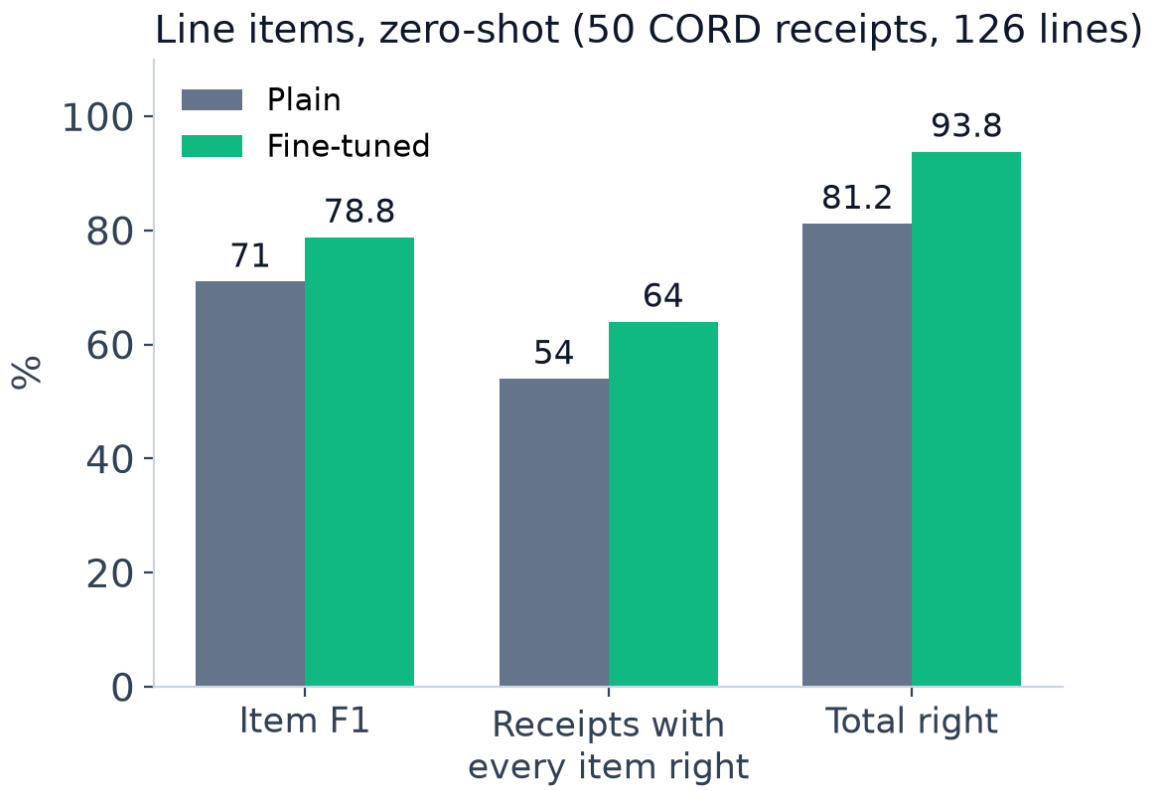
- They stayed identical across every checkpoint
- Our explanation (not verified): k/v projections in E2B's KV-shared layers get no gradient
- Smoke run said 12.5 s/step (warm-up); real 0.96 s/step

Gates passed before every run: budget applied (525-544 image tokens), pixel parity, labels contain the JSON answer, finite losses (first loss 2.56).

Bonus: vendor memory (RAG) and line items

BACKUP





276 shops from the training labels only; match threshold 0.85 fixed in advance. Fine-tuning had already learned most of what retrieval adds.

Zero-shot: SROIE has no line-item labels, so we scored on CORD. No forgetting; the fine-tuned model writes names in capitals and once shifted prices by a row.

APPENDIX

Appendix: settings and reproducibility

Setting	Value
Base model / load	unsloth/gemma-4-E2B-it, 16-bit LoRA (load_in_4bit = False)
LoRA	r 32, alpha 32, dropout 0; vision + language, attention + MLP; 59.8 M / 5.18 B (1.15 %)
Optimiser	adamw_8bit, lr 2e-4, cosine, warm-up 0.03, weight decay 0.001, grad norm 0.3
Batch / steps	1 x grad-accum 4; 5 smoke + 157 steps (1 epoch over 626 receipts)
Visual budget / prompt	max_soft_tokens 560 (525-544 tokens); image first, then instruction
Instruction	Extract the company, date, address and total from this receipt. Reply with JSON only.
Decoding	greedy, batch size 1, max_new_tokens 256 (every configuration)
Test set	100 of 361 SROIE test receipts, random.Random(0); keys in test_keys.json
Hardware / time	RTX PRO 6000 Blackwell (Colab G4, 95 GB); 2.5 min training, peak 13.0 GB
Speed (idle GPU)	plain 2.4 s, prompted 2.0 s, fine-tuned 3.0 s per receipt
Software	torch 2.11 cu130, transformers 5.5.0, unsloth 2026.10.8, trl 1.16.0, peft 0.21.2

Compute: trained on a Colab G4 for speed (2.5 min). Unsloth's docs put E2B LoRA at about 8-10 GB, so the recipe should also fit a free T4, more slowly; we did not test this.